In [1]:
```python
import pandas as pd
df = pd.read_csv(r'C:\Users\KHAN\Desktop\fraud data.csv')
print(df)
```

```
      transaction_id  user_id       amount transaction_type merchant_category  \
0               9608      363  4922.587542              ATM            Travel
1                456      692    48.018303               QR              Food
2               4747      587   136.881960           Online            Travel
3               6934      445    80.534719              POS          Clothing
4               1646      729   120.041158           Online           Grocery
...              ...      ...          ...              ...               ...
9995            1076      482    58.366442              POS          Clothing
9996            4995      904   139.502160              POS            Travel
9997            3485      527    71.012122           Online            Travel
9998            7922      771    21.031405               QR           Grocery
9999            6451      429    54.028632              ATM       Electronics

     country  hour  device_risk_score  ip_risk_score  is_fraud
0         TR    12           0.992347       0.947908         1
1         US    21           0.168571       0.224057         0
2         TR    14           0.296127       0.125058         0
3         TR    23           0.124801       0.159243         0
4         FR    16           0.098129       0.027542         0
...      ...   ...                ...            ...       ...
9995      DE    12           0.066366       0.086344         0
9996      DE    13           0.119014       0.285680         0
9997      TR     8           0.119204       0.262262         0
9998      UK    12           0.027088       0.295243         0
9999      DE    11           0.096438       0.042823         0

[10000 rows x 10 columns]
```

In [8]:
```python
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype 
---  ------             --------------  ----- 
 0   transaction_id     10000 non-null  int64 
 1   user_id            10000 non-null  int64 
 2   amount             10000 non-null  int64 
 3   transaction_type   10000 non-null  object
 4   merchant_category  10000 non-null  object
 5   country            10000 non-null  object
 6   hour               10000 non-null  int64 
 7   device_risk_score  10000 non-null  int64 
 8   ip_risk_score      10000 non-null  int64 
 9   is_fraud           10000 non-null  int64 
dtypes: int64(7), object(3)
memory usage: 781.4+ KB
None
```

In [11]:
```python
print(df.duplicated().sum())
```

```
0
```

In [2]:
```python
print(df['is_fraud'].unique())
```

```
[1 0]
```

In [3]:
```python
print(df['country'].unique())
```

```
['TR' 'US' 'FR' 'DE' 'UK' 'NG']
```

In [4]:
```python
df['country'] = df['country'].str.replace('TR', 'Turkey').str.replace('FR', 'France').str.replace('DE', 'Germany').st
```

In [5]:
```python
print(df['country'].unique())
```

```
['Turkey' 'US' 'France' 'Germany' 'UK' 'Nigeria']
```

In [8]:
```python
df['amount'] = df['amount'].astype(int)
```

In [9]:
```python
print(df['amount'].unique())
```

```
[ 4922    48   136    80   120    97   166    96    83    89   101   106
    55    99    94   133   157   202    88   115    63   113   131  4284
    76   138    17     1   154   103    54    91   135    64   117   174
    98   118   109   140    50   173    53    26    79     3   123    90
    49   127   159     5   139   122   165   132    47   107   108   124
    11    46   169   209  1148   184   952   104  2865    32    78    87
    74   200   111    84    75    36   102  2830    95   146    65    58
    82   190   128   116  2705    28   147   168   141    56   170   186
   149   126   152    22  2995   167    10  2400  1489    92    41   151
    93    70   177    27   137   125   171    19    25   176   158   134
   148    14   105    77   110   162  1927    23    86    51    40    71
    33   163   161    31    73    38   143  1110   182    81   129    29
    72  1094     7     6    59    44   164    21   114   121    67  3444
   172   100   242   160    60   238   150   155  2186    52    62   220
    13   206   130  1220   119    45    34   378   112   178    15  2920
   199   144    18   185    20    37   198  2011    57   145    61  4120
    30   191    43  2460   187  3980    66   201   244   181   189     9
   188  1223   203   665  2601  3036     8   223  4912    68   234   941
   183   193   180    69   153   175    39  2110    42   192   156  1343
    35  4603   142  1513  3842  1034  1074   222   567  3541  3856  3079
  1665  1853  1597  2983  1535   214  1772  1144    24   802   889  2894
   212   205  3578  1495    85  7801  2035   913    16   219   213   395
  2727  1157    12  2063     2  5450     4  1691  1919  1254   197   194
   659   207   196  2440   195   208  3512   228   210   224   240   221
  2879   860  2185  2355   986  1783  2273   787  2883   643   554   262
  3476  2389   232  1659   733  2016   204  1271  2665  4178   217  2557
  4555  1187  5955  1868  9647   405  1609   226  3471   612   179  3293
   837  1208  3238   225  2980  2869  3454   229  2182   402   235  1011
   822  2378  2758   422  1552  4946  4694  1615   650  4524  2227  1557
   249  2325  2364  1440 11628  1767   982  1598  3391  2924  3779   215
   652  2132  1228   937  1122   407  3462   231  5446   243  1766   684
  3199  1421  4031  1465  1024  1841  7504  2587  2237  6253  2330  1000
  2605  2149   227  2645  4150  2993  3149  2613  1638   216   691  7029
   518  2039  2313  2323  5230   562  2101   239   255   843  3464  2068
  1393   949   211  7653  2828   236  2792  5320   230   690   967  2923
   432  1681  1001   794  2053  2268  1054  4548  1881  2875   245  2473
   701  6256 10430  4868  2499   246  1645  2363  1312  1793  2217  3236
  1093  3342  2192  1209   773   815  3681  6770  2801  2459  1447  1091
  3214   237   812  1448  2541   274   605  3939  1684  1965  2718   807
  6336  1274  1345  2606  2675  1101  1136  4359  3213  1126 11085   906
  2416   248   623   870  2824   241  1586  2359  1169   932  1269  1298
  2381  4892  5187   791  3453  1711  1815  1411  1931  1480  4889  1624
```

```
 3178   638   277  1433  2909  1719   307  8065  1240   911  1026  3191
  269   832   624  1210  2085  5238  8893   295  1703  1234   591   762
 3139   218   400   842  1396   512   263  1739   539  1745  2056  2973
 1637  1068  3711  3725  1290  3788   703  4601  2767  3198  2836   769
 1909   984   257  1463  4542  2734  4340  3260  2328   796  2534  5975
  641  4765   303  2170   254  3010  1944  2799  1045  5182  3019  1382
  826   265   253  2661  1725  2990  1699   280  6427  2283  1758   461
 1455  2231  1381  1546]
```

In [10]:

```
print(df['transaction_type'].unique())
```

```
['ATM' 'QR' 'Online' 'POS']
```

In [4]:

```
df['device_risk_score'] = (df['device_risk_score']*10).astype(int)
df['ip_risk_score'] = (df['ip_risk_score']*10).astype(int)
```

In [7]:

```
print(df)
```

```
      transaction_id  user_id  amount transaction_type merchant_category  \
0               9608      363    4922              ATM            Travel   
1                456      692      48               QR              Food   
2               4747      587     136           Online            Travel   
3               6934      445      80              POS          Clothing   
4               1646      729     120           Online           Grocery   
...              ...      ...     ...              ...               ...   
9995            1076      482      58              POS          Clothing   
9996            4995      904     139              POS            Travel   
9997            3485      527      71           Online            Travel   
9998            7922      771      21               QR           Grocery   
9999            6451      429      54              ATM       Electronics   

      country  hour  device_risk_score  ip_risk_score  is_fraud  
0      Turkey    12                  9              9         1  
1          US    21                  1              2         0  
2      Turkey    14                  2              1         0  
3      Turkey    23                  1              1         0  
4      France    16                  0              0         0  
...       ...   ...                ...            ...       ...  
9995  Germany    12                  0              0         0  
9996  Germany    13                  1              2         0  
9997   Turkey     8                  1              2         0  
9998       UK    12                  0              2         0  
9999  Germany    11                  0              0         0  

[10000 rows x 10 columns]
```

In [12]:

```
def device_risk_score(m):
    if m < 3: return 'LOW'
    elif m < 6: return 'MEDIUM'
    else: return 'HIGH'
df['device_risk_score'] = df['device_risk_score'].apply(device_risk_score)
def ip_risk_score(m):
    if m < 3: return 'LOW'
    elif m < 6: return 'MEDIUM'
    else: return 'HIGH'
df['ip_risk_score'] = df['ip_risk_score'].apply(ip_risk_score)
```

In [16]:

```
print(df)
```

```
      transaction_id  user_id  amount transaction_type merchant_category  \
0               9608      363    4922              ATM            Travel
1                456      692      48               QR              Food
2               4747      587     136           Online            Travel
3               6934      445      80              POS          Clothing
4               1646      729     120           Online           Grocery
...              ...      ...     ...              ...               ...
9995            1076      482      58              POS          Clothing
9996            4995      904     139              POS            Travel
9997            3485      527      71           Online            Travel
9998            7922      771      21               QR           Grocery
9999            6451      429      54              ATM       Electronics

      country  hour device_risk_score ip_risk_score  is_fraud
0      Turkey    12              HIGH          HIGH         1
1          US    21               LOW           LOW         0
2      Turkey    14               LOW           LOW         0
3      Turkey    23               LOW           LOW         0
4      France    16               LOW           LOW         0
...       ...   ...               ...           ...       ...
9995  Germany    12               LOW           LOW         0
9996  Germany    13               LOW           LOW         0
9997   Turkey     8               LOW           LOW         0
9998       UK    12               LOW           LOW         0
9999  Germany    11               LOW           LOW         0

[10000 rows x 10 columns]
```

In [21]:

```
def fraud_loss(m):
    if m < 5000: return '<5K'
    elif m < 8000: return '<8K'
    elif m < 10000: return '<10K'
    else: return '10K>'
df['amount'] = df['amount'].apply(fraud_loss)
```

In [22]:

```
print(df)
```

```
      transaction_id  user_id amount transaction_type merchant_category  \
0               9608      363    <5K              ATM            Travel
1                456      692    <5K               QR              Food
2               4747      587    <5K           Online            Travel
3               6934      445    <5K              POS          Clothing
4               1646      729    <5K           Online           Grocery
...              ...      ...    ...              ...               ...
9995            1076      482    <5K              POS          Clothing
9996            4995      904    <5K              POS            Travel
9997            3485      527    <5K           Online            Travel
9998            7922      771    <5K               QR           Grocery
9999            6451      429    <5K              ATM       Electronics

      country  hour device_risk_score ip_risk_score  is_fraud
0      Turkey    12              HIGH          HIGH         1
1          US    21               LOW           LOW         0
2      Turkey    14               LOW           LOW         0
3      Turkey    23               LOW           LOW         0
4      France    16               LOW           LOW         0
...       ...   ...               ...           ...       ...
9995  Germany    12               LOW           LOW         0
9996  Germany    13               LOW           LOW         0
9997   Turkey     8               LOW           LOW         0
9998       UK    12               LOW           LOW         0
9999  Germany    11               LOW           LOW         0

[10000 rows x 10 columns]
```

In [19]:

```
import os
desktop = os.path.join(os.path.expanduser('~'), 'Desktop')
df.to_csv(r'C:\Users\KHAN\Desktop\Fraud_Data2.csv')
```